

Resumen Profundo Clase 4: El Arte de la Aproximación Numérica

Ing. Omar Cáceres

Modelado y Simulación

1. El Fundamento: ¿Por qué y Cómo aproximar?

El desafío central del modelado numérico consiste en calcular un valor estimativo y sumamente preciso ($\hat{I}$) de una integral definida $I = \int_a^b f(x)dx$ cuando la función $f(x)$ carece parcial o totalmente de una primitiva analítica viable (ej., integrales de funciones gaussianas de probabilidad).

La deducción fundamental del error entre este valor aproximado y el real recurre al **Lema del Valor Medio de la Integral**. Plantea que, en esencia, si f y g son continuas en un espectro y $g(x) \geq 0$, siempre existirá un punto intermedio libre y arbitrario $\xi \in [a, b]$ donde:

$$\int_a^b f(x)g(x)dx = f(\xi) \int_a^b g(x)dx \quad (1)$$

Con base a esto, evaluamos poligonales y calculamos el desvío exacto.

2. Regla del Rectángulo (Punto Medio)

Plantea resolver subintervalos usando siempre el centro de los mismos para modelar una figura constante recta pura (polinomio de Grado 0).

- **Aproximación Celda Unitaria** $[-h/2, h/2]$: $\int_{-h/2}^{h/2} f(x)dx \approx h \cdot f(0)$
- **Fórmula compuesta** (n particiones): $I_r = h \sum_{k=0}^{n-1} f(x_{k+1/2})$
- **Error Global Teórico (Truncamiento)**: $E_r = \frac{(b-a)h^2}{24} f''(\xi)$.

3. Regla del Trapecio

Abandona la constante pura y prefiere empalmar mediante una recta oblicua (secante de Grado 1) los pares de valores que engloban a cada subintervalo continuo evaluado.

- **Fórmula compuesta**:

$$I_t = \frac{h}{2} \left[f(x_0) + 2 \sum_{k=1}^{n-1} f(x_k) + f(x_n) \right] \quad (2)$$

- **Error Global Teórico**: $E_t = -\frac{(b-a)h^2}{12} f''(\xi)$.

Detalle Técnico Clave: Una intuición visual sugiere que empalmar con un techo sesgado en trapecio es inherentemente "mejor" que cortar a la mitad usando cajas llanas cuadradas (rectángulo). Contradictoriamente, el factor limitante $\frac{1}{24}$ en el Rectángulo y $\frac{1}{12}$ en el Trapecio evidencian teóricamente que el modelo Rectangular suele producir la mitad del error neto bajo un h equitativo, si bien ambos comparten el orden global de convergencia $\mathcal{O}(h^2)$.

4. Regla de Simpson 1/3 (La Malla Parabólica)

Asciende al nivel Cuadrático de Newton-Cotes. Entraña envolver tríos sucesivos de puntos (x_i, x_{i+1}, x_{i+2}) con un contorno de molde de parábola de subgrado 2.

- **Restricción Metodológica Ineludible:** El conteo puro de subintervalos n debe ser rigurosamente un número par.
- **Fórmula compuesta generalizada:**

$$I_s = \frac{h}{3} \left[f(x_0) + 4 \sum_{i \text{ impar}} f(x_i) + 2 \sum_{i \text{ par}} f(x_i) + f(x_n) \right] \quad (3)$$

- **Orden Crítico de Convergencia:** $\mathcal{O}(h^4)$. Un recorte de h por dos contrae la varianza al valor de $\frac{1}{16}$ partes del error primario original.
- **Error Global:** $E_{simp} = -\frac{(b-a)h^4}{180} f^{(4)}(\xi)$.
Importante: Al ser directamente dependiente aritméticamente de la 4ta Derivada general de la ecuación principal de base, este método modela y otorga exactitud pura y neta a las funciones primitivas cúbicas plenas convencionales ($f(x) \propto x^3$).

5. Regla de Simpson 3/8 (Cúbica)

Se expande interpolando no 3, sino 4 nodos con un polinomio cúbico puro (Grado 3).

- **Restricción:** $n \pmod{3} \equiv 0$ (n es un múltiplo de 3).
- **Fórmula por Segmento:** $\approx \frac{3h}{8} [f(a) + 3f(x_1) + 3f(x_2) + f(b)]$
- **Error Global Teórico:** Exactamente de orden $\mathcal{O}(h^4)$, cuantificado operativamente en $-\frac{(b-a)h^4}{80} f^{(4)}(\xi)$.

6. Estimación Estructural de Ingeniería: Parámetro h_{max}

No podemos operar asumiendo límites intangibles. Si imponemos o requerimos por contrato de modelo una *Tolerancia Estructural Maximizada* tope equivalente a τ (p.ej., $\tau = 10^{-5}$):

Para Simpson 1/3, si asilamos y acotamos ξ ubicando el peor de los casos máximos de nuestra respectiva cuota de derivada, definiéndolo como $M_4 = \max_{x \in [a,b]} |f^{(4)}(x)|$:

$$|E| = \frac{b-a}{180} \cdot h^4 \cdot M_4 \leq \tau \quad (4)$$

Despejamos el parámetro precalculable:

$$h_{max} = \sqrt[4]{\frac{180 \cdot \tau}{(b-a) \cdot M_4}} \quad (5)$$

Al configurar programáticamente una partición unánime $h \leq h_{max}$ nuestro script estará provisto de una garantía absoluta paramétrica asumiendo flotantes teóricos.

7. Dilema Analítico: Truncamiento Matemático vs Limitante Físico de Redondeo

La cota anterior suponía operar calculadoras irreales teóricas matemáticas de precisión ∞ . Al traspasar los resultados al microprocesador:

- **Error de Truncamiento:** Se acopla a las ecuaciones $\mathcal{O}(h^k)$ presentadas, mermando vertiginosamente al reducir dramáticamente el paso h .
- **Error de Redondeo Finito Flotante:** Es inversamente proporcional al tamaño de h . Interpoliar con cortes infinitesimales minúsculos ($h \rightarrow 0$) infiere procesar brutalmente sumatorias en bucle por ciclos for. Cada una de ellas infiere e inyecta diminutas podredumbres de falta de memoria (coma flotante en bytes acotados). Hay un "Valle Óptimo.^a partir del cual refinar h degrada irreversiblemente a todo el cálculo puro final.

8. Implementación en Script Puro de Python (NumPy Array Ops)

```
import numpy as np

# Configurar hiper-parametros
n = 120 # Subintervalos elegidos
h = (b - a) / n
x = np.linspace(a, b, n+1)
y = func(x)

# Regla del Trapecio
int_t = (h / 2) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])

# Regla de Simpson 1/3 (Garantizado un N = par localmente)
# Extrae el slice desde el index 1 y da saltos de a 2.
int_s = (h / 3) * (y[0] + y[-1] + 4 * np.sum(y[1:-1:2]) + 2 * np.sum(y[2:-2:2]))

# Regla de Simpson 3/8 (Garantizado un N = % 3)
int_s3 = (3 * h / 8) * (y[0] + y[-1] + 3 * np.sum(y[1:-1:3]))
```

```
+ 3 * np.sum(y[2:-1:3]) + 2 * np.sum(y  
[3:-2:3]))
```